

Hardware Setup and Data Flow

Raspberry Pi 5 + Infineon SLB9672 TPM 2.0

Connection Details

The TPM2.0 module needs to be interfaced with raspberry pi using SPI protocol. The TPM module to raspberry pi GPIO pin mapping is provided in the below table.

TPM pin	Signal	Raspberry Pi pin	Action
1	3.3 V	1	Connect
2	CS	26 / GPIO7 / CE1	Connect — critical
3	MISO	21 / GPIO9	Connect
4	MOSI	19 / GPIO10	Connect
5	Reserved	—	Do not connect
6	SCLK	23 / GPIO11	Connect
7	GND	6	Connect
8	RESET	7 / GPIO4	Connect
9	Reserved	—	Do not connect
10	No pin	—	No connection
11	Reserved	—	Do not connect
12	IRQ	—	Leave disconnected initially

Linux Architecture

Raspberry Pi firmware applies Device Tree overlays from config.txt before passing the resolved Device Tree to Linux. Infineon confirms that the Raspberry Pi tpm-slb9670 overlay can be used with SLB9672.

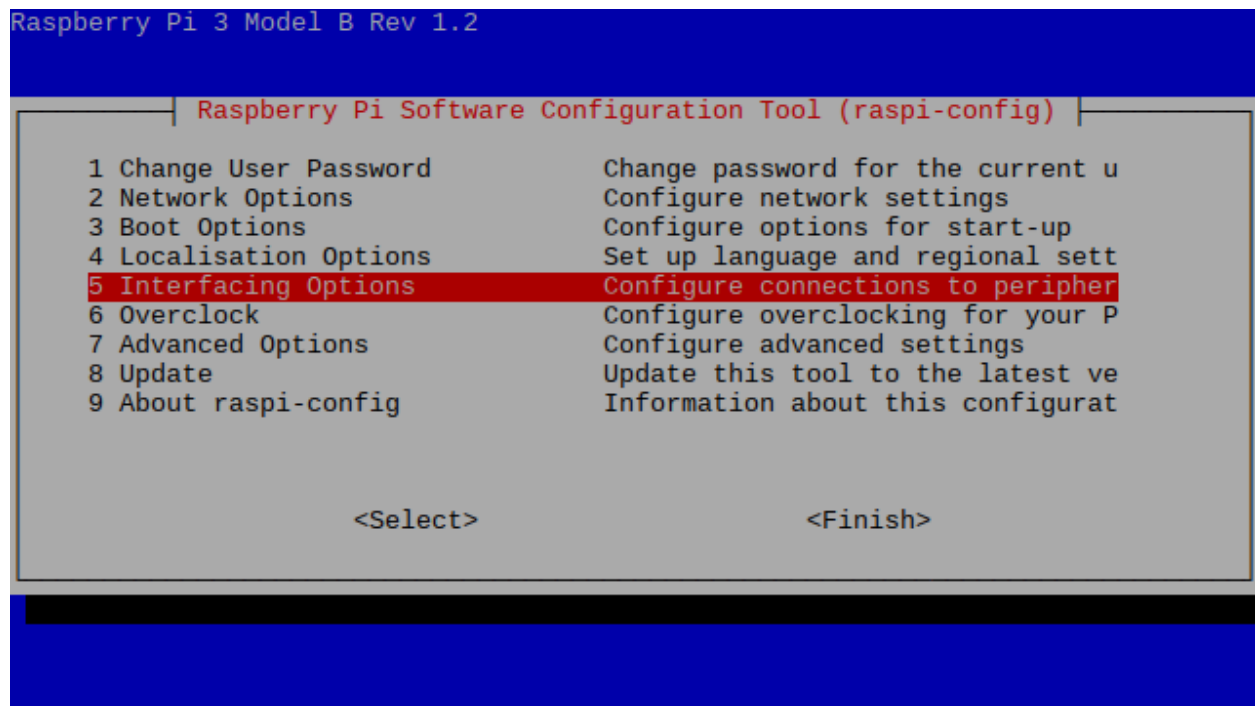
For connecting the TPM2.0 module with raspberry pi, we first need to enable SPI interface in the raspberry pi configurations.

Step 1 – Enable SPI Interface

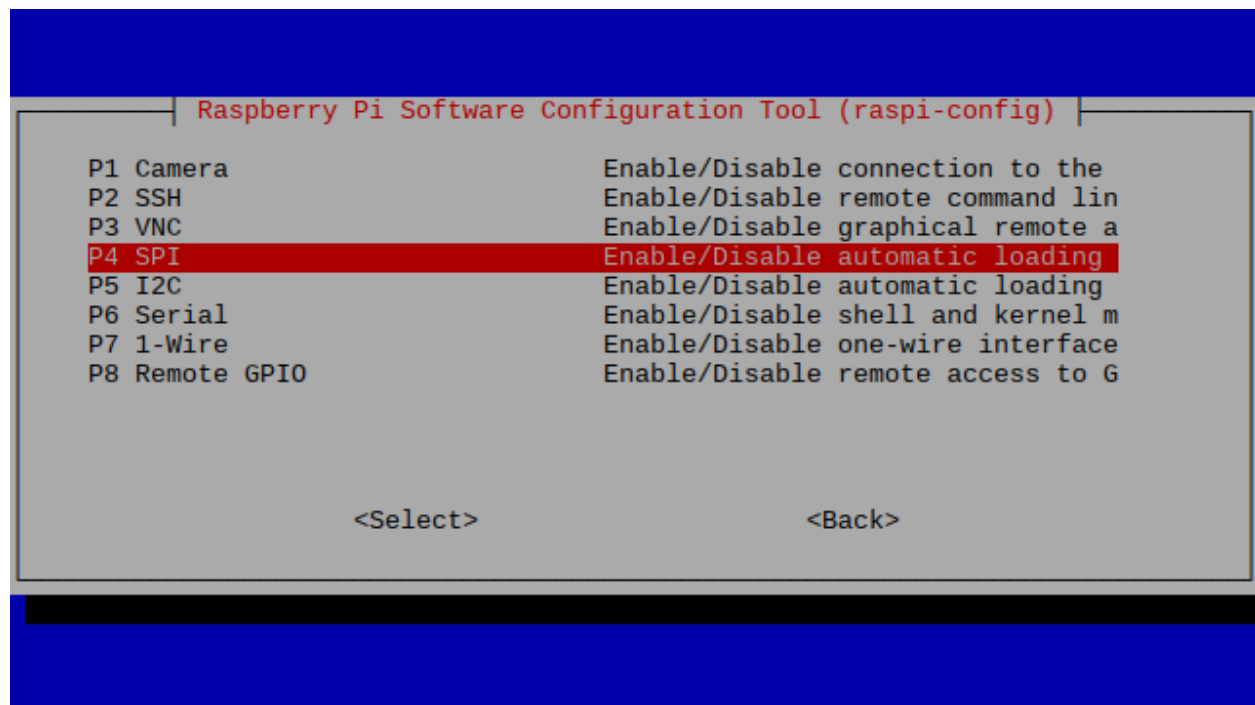
The default Raspbian image disables SPI by default so before you can use it the interface must be enabled. This can be done using either of three methods. I'll describe all methods but the first one is probably easier and quicker.

From the command line or Terminal window start by running the following command :
`sudo raspi-config`

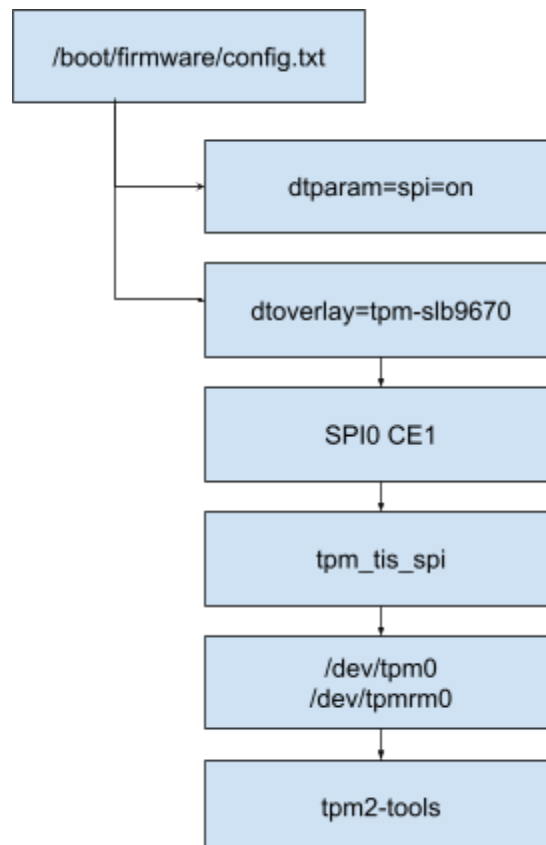
This will launch the raspi-config utility. Select “Interfacing Options” :



Highlight the “SPI” option and activate “<Select>”.



Select and activate “<Yes>” :



Recommended config.txt

```
# Raspberry Pi 5 + Infineon SLB9672
```

```
dtparam=spi=on  
dtparam=audio=on
```

```
camera_auto_detect=1  
display_auto_detect=1  
auto_initramfs=1
```

```
dtoverlay=vc4-kms-v3d  
max_framebuffers=2  
disable_fw_kms_setup=1  
arm_64bit=1  
disable_overscan=1  
arm_boost=1
```

```
[cm4]  
otg_mode=1
```

```
[cm5]  
dtoverlay=dwc2,dr_mode=host
```

```
[pi5]  
dtoverlay=nospi10
```

```
[all]  
dtparam=uart0=on
```

```
# SLB9672: Raspberry Pi's standard SLB9670 overlay  
dtoverlay=tpm-slb9670
```

During TPM bring-up, remove/comment out `dtoverlay=w1-gpio` because the TPM RESET line is on GPIO4. This avoids a possible GPIO4 ownership conflict while debugging. Re-enable 1-Wire later only after selecting a non-conflicting GPIO.

Troubleshooting History

Observed	Likely cause	Fix	Outcome
Boot error code 7: kernel/device-tree not found	Original SD/boot files became inconsistent/corrupt; TPM did not itself edit config.txt	Re-flashed Raspberry Pi OS	Boot restored
No /dev/spidev*	SPI disabled	Enable SPI; use <code>dtparam=spi=on;</code> reboot	spidev0.0/.1 appeared

No /dev/tpm*	TPM not described in Device Tree	Add dtoverlay=tpm-slb9 670	Kernel began probing TPM
tpm_tis_spi spi0.1 probe failed -110	Driver probed CE1 but physical CS was initially on CE0/pin 24	Move TPM CS to pin 26/CE1	/dev/tpm0 and /dev/tpmrm0 appeared
tpm2_getcap permission denied	TPM nodes were root-only	Use sudo for initial testing; later configure access policy if required	Root could access device
TPM not initialized by TPM2_Startup	TPM returned TPM_RC_INITIALIZE	Do not repeatedly force Startup; validate stable boot/startup sequence first	Communication was established
tpm2_startup -c timed out	Command-level timeout remained; reset/GPIO/startup sequencing needed further validation	Remove w1-gpio during TPM bring-up and retest	Last validation step

Bring-up Procedure

Power off before changing wiring.

Connect the TPM exactly as in Section 2, especially CS → physical pin 26/CE1.

Back up config: `sudo cp /boot/firmware/config.txt /boot/firmware/config.txt.bak`

Replace config.txt with Section 4 and omit w1-gpio during TPM bring-up.

Reboot: `sudo reboot`

Check: `ls -l /dev/tpm*`

Check: `dmesg | grep -i -E 'tpm|spi|slb'`

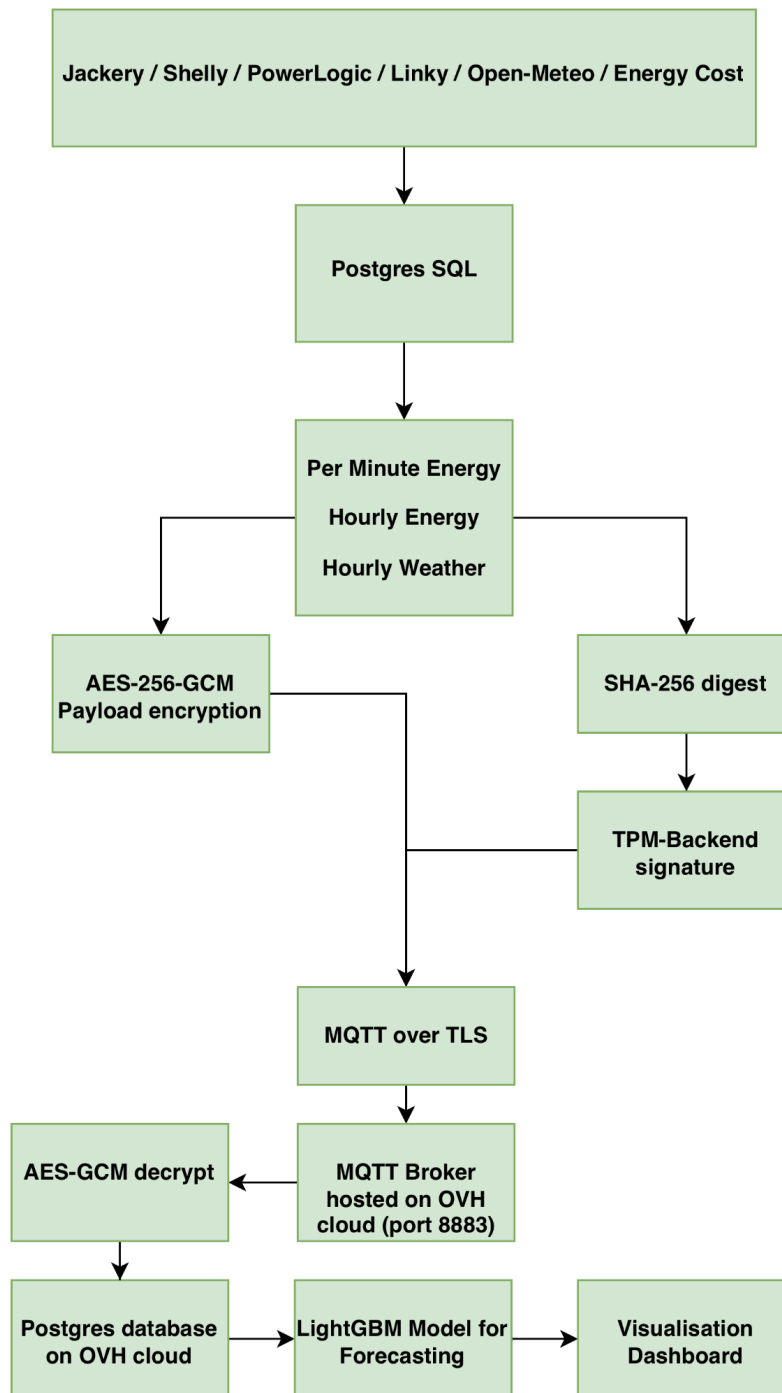
Install tools: `sudo apt update && sudo apt install tpm2-tools`

Test: `sudo tpm2_getcap properties-fixed`

Do not repeatedly run `tpm2_startup -c` unless startup state is specifically being diagnosed.

Secure MQTT Architecture

Uses two layers: MQTT over TLS for secure transport, plus application-layer encryption when the payload must remain confidential from the broker or from intermediate storage. MQTT 5 explicitly describes TLS as a secured transport and also permits independent application-message encryption.



Payload Encryption

For sensor payloads, AES-256-GCM is a suitable authenticated-encryption construction. Generate a fresh unique 96-bit nonce for every encryption under a given AES key.

```
C, TAG = AES-256-GCM-Encrypt(  
    key = DEK,  
    nonce = fresh_96_bit_nonce,  
    plaintext = sensor_payload,  
    AAD = device_id || timestamp || schema_version  
)
```

MQTT payload:

```
{  
  "version": 1,  
  "device_id": "device-001",  
  "key_id": "device-001-k1",  
  "nonce": "<base64>",  
  "ciphertext": "<base64>",  
  "tag": "<base64>"  
}
```

The broker can route and store the encrypted payload without possessing the application decryption key. Payload encryption does not hide MQTT topic names or other MQTT metadata.

Layer	Mechanism	Protects	Does not protect
Transport	MQTT over TLS	Network confidentiality, transport integrity, broker authentication	Payload after it reaches the broker; MQTT topic metadata from application-layer encryption
Application payload	AES-256-GCM	End-to-end payload confidentiality and authenticated integrity	Topic name and other MQTT metadata
Device identity	TPM-backed private key	Private key extraction resistance, device signing/authentication	Bulk encryption of sensor streams
MQTT authorization	Per-device topic ACLs	Which devices may publish/subscribe to which topics	Compromised authorized device's own payloads
FL validation	Signature + round/sequence checks	Update authenticity and replay resistance	Malicious behavior by a legitimately authenticated client

MQTT Message Payload and Topics:

```
energy/<device_id>/telemetry  
energy/<device_id>/weather  
energy/<device_id>/cost  
energy/<device_id>/status
```

```
federated/<device_id>/gradient  
federated/<device_id>/model-update  
federated/<device_id>/ack
```

```
device/<device_id>/command  
device/<device_id>/configuration
```

MQTT Topic

```
|  
├─ energy/<device_id>/minute  
├─ energy/<device_id>/hourly  
├─ energy/<device_id>/weather  
└─ federated/<device_id>/gradient
```



Secure MQTT Envelope

version
device_id
message_type
timestamp
key_id
algorithm
nonce
ciphertext
authentication_tag
signature

Plain text payload before encryption:

```
{  
  
  "device_id": "shelly_001",  
  
  "device_name": "Living Room",  
  
  "timestamp": "2026-09-22T21:15:00",  
  
  "current_a": 2.431,  
  
  "voltage_v": 231.7,  
  
  "power_w": 523.4,  
  
  "energy_minute_wh": 8.723  
  
}
```

```
{  
  "timestamp": "2026-09-22T20:00:00",  
  "temperature_c": 28.4,  
  "humidity_percent": 73.2,  
  "wind_speed_ms": 3.7,  
  "precipitation_mm": 0.0,  
  "condition": "clear",  
  "ghi_wm2": 542.3  
}
```

After wrapping the payloads:

```
{  
  "version": 1,  
  "device_id": "gateway_001",  
  "message_id": "01JXYZ...",  
  "sequence": 18452,  
  "message_type": "energy_minute",  
  "timestamp": "2026-09-22T21:15:00Z",  
  "key_id": "gateway_001_key_01",  
  "algorithm": "AES-256-GCM",  
  "nonce": "...",  
  "ciphertext": "...",  
  "tag": "...",  
  "signature_algorithm": "ECDSA-SHA256",  
  "signature": "..."  
}
```

Requirements from the devops team:

Dependency	Status	Requirement
Broker hostname / DNS	Required	DevOps must provide the production/dev EMQX FQDN or IP. Do not hard-code an IP in the Raspberry Pi application.
MQTT TLS port	Required	Use 8883 for the protected MQTT connection shown in the supplied architecture. Port 1883 should be restricted to controlled development/internal use if retained.
Broker CA certificate	Required	Provide the CA chain needed by the Raspberry Pi to validate the EMQX server certificate.
Client authentication	Required	Define whether the edge uses mTLS client certificates, username/password, or another EMQX-supported mechanism. For production, the identity and credential lifecycle must be specified.
Client ID convention	Required	Define a unique gateway/device ID format, e.g. h2s-edge-<site>-<gateway>. It must remain stable enough for broker ACL/session management.
Topic namespace	Required	Backend team must freeze the topic hierarchy before implementation. Suggested: h2s/v1/telemetry/<gateway_id>/energy/minute, h2s/v1/telemetry/<gateway_id>/energy/hourly, h2s/v1/telemetry/<gateway_id>/weather, h2s/v1/fl/<gateway_id>/gradient.
Topic ACL	Required	Each gateway should publish only to its assigned namespace and subscribe only where required. The gateway should not have wildcard write access.
QoS policy	Required	Agree QoS per message type. Recommended starting point: QoS 1 for telemetry requiring delivery, with application-level message_id/sequence for duplicate detection.
Retained-message policy	Required	Define whether any telemetry/status topic may be retained. Do not retain high-rate measurements unless explicitly required.
Payload size limit	Required	Define the maximum encrypted MQTT payload accepted by EMQX and

		downstream services, especially for federated-learning gradients.
Message schema/versioning	Required	Freeze JSON envelope fields, units, timestamp format, schema version, and backward-compatibility rules.
Application-layer encryption	Required	Confirm that the edge payload is encrypted before MQTT publication. Recommended: AES-256-GCM with a unique nonce per message.
Signing / device identity	Required	Define the signature algorithm and verification endpoint. The Raspberry Pi TPM should protect the device signing private key; the backend must maintain the corresponding public-key/device identity.
Key provisioning	Required	Define initial provisioning, key rotation, revocation, recovery, and device replacement. TPM ownership/provisioning must be part of the manufacturing/onboarding process.
Replay protection	Required	Backend must accept and validate message_id + sequence + timestamp (and/or an equivalent monotonic counter) and define the replay window.
Clock synchronization	Required	NTP/chrony or another trusted time source is required if timestamp freshness/replay protection is enforced.
Offline buffering	Required	Define how the Raspberry Pi queues messages when EMQX is unavailable, maximum queue size, persistence location, retry/backoff, and expiry policy.
Downstream consumer	Required	Confirm whether agent-sync is the first consumer of edge MQTT messages and define the exact transformation from edge topic/payload to backend events.
Observability	Required	Define which EMQX/client metrics and logs are collected by Prometheus/Grafana/Loki/Alertmanager, as shown in the supplied architecture.
Firewall / network path	Required	The edge should normally initiate outbound TCP/TLS to the broker. Define DNS, firewall, proxy/VPN requirements and whether inbound connections to the Raspberry Pi are prohibited.

Questions for Devops:

1. What is the EMQX FQDN for DEV, TEST and PROD?
2. Is 8883 mandatory for all edge-to-cloud MQTT traffic?
3. Which CA certificate and client certificate/credential should each Raspberry Pi receive?
4. What is the client-ID naming convention?
5. What exact topic hierarchy and ACL should a gateway receive?
6. Which topics are consumed by agent-sync, and what is the expected downstream mapping?
7. What QoS, keepalive, session expiry and retained-message policy should be used?
8. What is the maximum MQTT payload size for telemetry and federated-learning gradients?
9. Who owns message-schema versioning?
10. How are device keys provisioned, rotated, revoked and recovered?
11. Which public-key registry will the backend use to verify TPM-backed signatures?
12. What replay window and sequence-number policy will the backend enforce?
13. How long should the Raspberry Pi buffer data during broker/network outages?
14. Which Prometheus/Loki metrics and alerts are mandatory for the edge MQTT connection?
15. What are the DEV/TEST/PROD network firewall/VPN/DNS requirements?